BIBLIOGRAFIE

1. https://www.fortinet.com/resources/cyberglossary/proxy-server
2. https://datatracker.ietf.org/doc/html/rfc3849
3. https://www.ruby-lang.org/en/documentation/ruby-from-other-languages/to-ruby-from-python/
4. https://levelup.gitconnected.com/tips-to-learn-ruby-with-a-python-background-2ca806a891ba

COD LABORATOR:

1) **Programul fără operații în proxy:**
   a) **Python:**

Pornesc un simplu server pe 8080 ca sa pot testa programul:

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
127.0.0.1 - - [13/Dec/2024 13:25:10] "GET / HTTP/1.1" 200 -
```

Rulez programul pentru acest server:

Trimit o cerere prin **curl** de la un client**:**

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ python3 proxy.py 127.0.0.1 9000 127.0.0.1 8080 True
[*] Ascult pe 127.0.0.1:9000
```

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ curl -x 127.0.0.1:9000 http://example.com
<!DOCTYPE HTML>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <title>Error response</title>
    </head>
    <body>
        <h1>Error response</h1>
        <p>Error code: 404</p>
        <p>Message: File not found.</p>
        <p>Error code explanation: 404 - Nothing matches the given URI.</p>
    </body>
</html>
(.venv) diana@debian:~/PycharmProjects/Lab11$
```

Proxy.py:

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ python3 proxy.py 127.0.0.1 9000 127.0.0.1 8080 True
[*] Ascult pe 127.0.0.1:9000
[==>] Primesc date de la 127.0.0.1:49800

[==>] Am primit 123 octeti de la clientul local.
0000  47 45 54 20 68 74 74 70 3A 2F 2F 65 78 61 6D 70   GET http://examp
0010  6C 65 2E 63 6F 6D 2F 20 48 54 54 50 2F 31 2E 31   le.com/ HTTP/1.1
0020  0D 0A 48 6F 73 74 3A 20 65 78 61 6D 70 6C 65 2E   ..Host: example.
0030  63 6F 6D 0D 0A 55 73 65 72 2D 41 67 65 6E 74 3A   com..User-Agent:
0040  20 63 75 72 6C 2F 37 2E 38 38 2E 31 0D 0A 41 63    curl/7.88.1..Ac
0050  63 65 70 74 3A 20 2A 2F 2A 0D 0A 50 72 6F 78 79   cept: */*..Proxy
0060  2D 43 6F 6E 6E 65 63 74 69 6F 6E 3A 20 4B 65 65   -Connection: Kee
0070  70 2D 41 6C 69 76 65 0D 0A 0D 0A                  p-Alive....
[==>] Trimit catre exterior
[<==] Am primit 520 octeti din afara
0000  48 54 54 50 2F 31 2E 30 20 34 30 34 20 46 69 6C   HTTP/1.0 404 Fil
0010  65 20 6E 6F 74 20 66 6F 75 6E 64 0D 0A 53 65 72   e not found..Ser
0020  76 65 72 3A 20 53 69 6D 70 6C 65 48 54 54 50 2F   ver: SimpleHTTP/
0030  30 2E 36 20 50 79 74 68 6F 6E 2F 33 2E 31 31 2E   0.6 Python/3.11.
0040  32 0D 0A 44 61 74 65 3A 20 46 72 69 2C 20 31 33   2..Date: Fri, 13
0050  20 44 65 63 20 32 30 32 34 20 31 31 3A 33 33 3A    Dec 2024 11:33:
0060  31 31 20 47 4D 54 0D 0A 43 6F 6E 6E 65 63 74 69   11 GMT..Connecti
0070  6F 6E 3A 20 63 6C 6F 73 65 0D 0A 43 6F 6E 74 65   on: close..Conte
0080  6E 74 2D 54 79 70 65 3A 20 74 65 78 74 2F 68 74   nt-Type: text/ht
0090  6D 6C 3B 63 68 61 72 73 65 74 3D 75 74 66 2D 38   ml;charset=utf-8
00A0  0D 0A 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68   ..Content-Length
00B0  3A 20 33 33 35 0D 0A 0D 0A 3C 21 44 4F 43 54 59   : 335....<!DOCTY
00C0  50 45 20 48 54 4D 4C 3E 0A 3C 68 74 6D 6C 20 6C   PE HTML>.<html l
00D0  61 6E 67 3D 22 65 6E 22 3E 0A 20 20 20 20 3C 68   ang="en">.    <h
00E0  65 61 64 3E 0A 20 20 20 20 20 20 20 20 3C 6D 65   ead>.        <me
00F0  74 61 20 63 68 61 72 73 65 74 3D 22 75 74 66 2D   ta charset="utf-
0100  38 22 3E 0A 20 20 20 20 20 20 20 20 3C 74 69 74   8">.        <tit
0110  6C 65 3E 45 72 72 6F 72 20 72 65 73 70 6F 6E 73   le>Error respons
0120  65 3C 2F 74 69 74 6C 65 3E 0A 20 20 20 20 3C 2F   e</title>.    </
0130  68 65 61 64 3E 0A 20 20 20 20 3C 62 6F 64 79 3E   head>.    <body>
0140  0A 20 20 20 20 20 20 20 20 3C 68 31 3E 45 72 72   .        <h1>Err
0150  6F 72 20 72 65 73 70 6F 6E 73 65 3C 2F 68 31 3E   or response</h1>
0160  0A 20 20 20 20 20 20 20 20 3C 70 3E 45 72 72 6F   .        <p>Erro
0170  72 20 63 6F 64 65 3A 20 34 30 34 3C 2F 70 3E 0A   r code: 404</p>.
```

```
0130  68 65 61 64 3E 0A 20 20 20 20 3C 62 6F 64 79 3E   head>.    <body>
0140  0A 20 20 20 20 20 20 20 20 3C 68 31 3E 45 72 72   .        <h1>Err
0150  6F 72 20 72 65 73 70 6F 6E 73 65 3C 2F 68 31 3E   or response</h1>
0160  0A 20 20 20 20 20 20 20 20 3C 70 3E 45 72 72 6F   .        <p>Erro
0170  72 20 63 6F 64 65 3A 20 34 30 34 3C 2F 70 3E 0A   r code: 404</p>.
0180  20 20 20 20 20 20 20 20 3C 70 3E 4D 65 73 73 61           <p>Messa
0190  67 65 3A 20 46 69 6C 65 20 6E 6F 74 20 66 6F 75   ge: File not fou
01A0  6E 64 2E 3C 2F 70 3E 0A 20 20 20 20 20 20 20 20   nd.</p>.
01B0  3C 70 3E 45 72 72 6F 72 20 63 6F 64 65 20 65 78   <p>Error code ex
01C0  70 6C 61 6E 61 74 69 6F 6E 3A 20 34 30 34 20 2D   planation: 404 -
01D0  20 4E 6F 74 68 69 6E 67 20 6D 61 74 63 68 65 73    Nothing matches
01E0  20 74 68 65 20 67 69 76 65 6E 20 55 52 49 2E 3C    the given URI.<
01F0  2F 70 3E 0A 20 20 20 20 3C 2F 62 6F 64 79 3E 0A   /p>.    </body>.
0200  3C 2F 68 74 6D 6C 3E 0A                           </html>.
[<==] Trimis local.
[*] Nu mai sunt date, inchid conexiunea.
```

b) **C#:**
build: “dotnet build”

run: “dotnet run -- 127.0.0.1 9000 127.0.0.1 8080 True”

```
dotnet run -- 127.0.0.1 9000 127.0.0.1 8080 True
[*] Ascult pe 127.0.0.1:9000
[==>] Primesc date de la 127.0.0.1:41148
[==>] Am primit 123 octeti de la clientul local.
0000  47 45 54 20 68 74 74 70 3A 2F 2F 65 78 61 6D 70   GET http://examp
0010  6C 65 2E 63 6F 6D 2F 20 48 54 54 50 2F 31 2E 31   le.com/ HTTP/1.1
0020  0D 0A 48 6F 73 74 3A 20 65 78 61 6D 70 6C 65 2E
Host: example.
0030  63 6F 6D 0D 0A 55 73 65 72 2D 41 67 65 6E 74 3A   com
User-Agent:
0040  20 63 75 72 6C 2F 37 2E 38 38 2E 31 0D 0A 41 63    curl/7.88.1
Ac
0050  63 65 70 74 3A 20 2A 2F 2A 0D 0A 50 72 6F 78 79   cept: */*
Proxy
0060  2D 43 6F 6E 6E 65 63 74 69 6F 6E 3A 20 4B 65 65   -Connection: Kee
0070  70 2D 41 6C 69 76 65 0D 0A 0D 0A                  p-Alive
```

```
0080  20 3C 62 6F 64 79 3E 0A 20 20 20 20 20 20 20 20    <body>

0090  3C 68 31 3E 45 72 72 6F 72 20 72 65 73 70 6F 6E   <h1>Error respon
00A0  73 65 3C 2F 68 31 3E 0A 20 20 20 20 20 20 20 20   se</h1>

00B0  3C 70 3E 45 72 72 6F 72 20 63 6F 64 65 3A 20 34   <p>Error code: 4
00C0  30 34 3C 2F 70 3E 0A 20 20 20 20 20 20 20 20 3C   04</p>
        <
00D0  70 3E 4D 65 73 73 61 67 65 3A 20 46 69 6C 65 20   p>Message: File
00E0  6E 6F 74 20 66 6F 75 6E 64 2E 3C 2F 70 3E 0A 20   not found.</p>

00F0  20 20 20 20 20 20 20 3C 70 3E 45 72 72 6F 72 20          <p>Error
0100  63 6F 64 65 20 65 78 70 6C 61 6E 61 74 69 6F 6E   code explanation
0110  3A 20 34 30 34 20 2D 20 4E 6F 74 68 69 6E 67 20   : 404 - Nothing
0120  6D 61 74 63 68 65 73 20 74 68 65 20 67 69 76 65   matches the give
0130  6E 20 55 52 49 2E 3C 2F 70 3E 0A 20 20 20 20 3C   n URI.</p>
    <
0140  2F 62 6F 64 79 3E 0A 3C 2F 68 74 6D 6C 3E 0A      /body>
</html>

[*] NU mai sunt date inchid conexiunea.
```

C/C++

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

class Proxy
{
    public static void HexDump(byte[] src, int length = 16)
    {
        for (int i = 0; i < src.Length; i += length)
        {
            var segment = src.AsSpan(i, Math.Min(length, src.Length - i));
            var hexa = BitConverter.ToString(segment.ToArray()).Replace("-", "
");
            var text =
Encoding.ASCII.GetString(segment.ToArray()).Replace('\0', '.');
            Console.WriteLine($"{i:X4}  {hexa,-48}  {text}");
        }
    }

    public static byte[] ReceiveData(Socket connection)
    {
        byte[] buffer = new byte[4096];
        int received = 0;
        try
        {
            connection.ReceiveTimeout = 2000;
            received = connection.Receive(buffer);
        }
        catch (SocketException)
        {
            //timeout or no data available
        }
        return buffer[..received];
    }

    public static byte[] RequestHandler(byte[] buffer)
    {
        // modificari asupra pachetelor
        return buffer;
    }
```

```
    public static byte[] ResponseHandler(byte[] buffer)
    {
        // modificari asupra pachetelor
        return buffer;
    }

    public static void ProxyHandler(Socket clientSocket, string remoteHost, int
remotePort, bool receiveFirst)
    {
        var remoteSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
        remoteSocket.Connect(remoteHost, remotePort);

        if (receiveFirst)
        {
            var remoteBuffer = ReceiveData(remoteSocket);
            if (remoteBuffer.Length > 0)
            {
                HexDump(remoteBuffer);
                remoteBuffer = ResponseHandler(remoteBuffer);
                clientSocket.Send(remoteBuffer);
            }
        }

        while (true)
        {
            var localBuffer = ReceiveData(clientSocket);
            if (localBuffer.Length > 0)
            {
                Console.WriteLine($"[==>] Am primit {localBuffer.Length} octeti
de la clientul local.");
                HexDump(localBuffer);

                localBuffer = RequestHandler(localBuffer);
                remoteSocket.Send(localBuffer);
            }

            var remoteBuffer = ReceiveData(remoteSocket);
            if (remoteBuffer.Length > 0)
            {
                Console.WriteLine($"[<==] Am primit {remoteBuffer.Length}
octeti din afara.");
                HexDump(remoteBuffer);
```

```
                remoteBuffer = ResponseHandler(remoteBuffer);
                clientSocket.Send(remoteBuffer);
            }

            if (localBuffer.Length == 0 && remoteBuffer.Length == 0)
            {
                Console.WriteLine("[*] NU mai sunt date inchid conexiunea.");
                clientSocket.Close();
                remoteSocket.Close();
                break;
            }
        }
    }

    public static void ServerLoop(string localHost, int localPort, string
remoteHost, int remotePort, bool receiveFirst)
    {
        var server = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp);
        server.Bind(new IPEndPoint(IPAddress.Parse(localHost), localPort));
        server.Listen(5);

        Console.WriteLine($"[*] Ascult pe {localHost}:{localPort}");

        while (true)
        {
            var clientSocket = server.Accept();
            Console.WriteLine($"[==>] Primesc date de la
{clientSocket.RemoteEndPoint}");

            var proxyThread = new Thread(() =>
            {
                ProxyHandler(clientSocket, remoteHost, remotePort,
receiveFirst);
            });
            proxyThread.Start();
        }
    }

    public static void Main(string[] args)
    {
        if (args.Length != 5)
        {
```

```
            Console.WriteLine("Utilizare: proxy [localhost] [localport]
[remotehost] [remoteport] [receive_first]");
            Console.WriteLine("De exemplu: proxy 127.0.0.1 9000 10.12.132.19
9000 True");
            return;
        }

        var localHost = args[0];
        var localPort = int.Parse(args[1]);
        var remoteHost = args[2];
        var remotePort = int.Parse(args[3]);
        var receiveFirst = bool.Parse(args[4]);

        ServerLoop(localHost, localPort, remoteHost, remotePort, receiveFirst);
    }
}
```

Tema

Utilizați scapy pentru a completa cele două zone de modificare a fluxului prevazute în program

Un ex de utilizare a scapy

2)

```
hostinput = str(input("Dati un IP sau un FQDN: "))
```

Python

```python
# schimbarea oricarei cereri destinate unei alte masini
def request_handler(buffer):
    # se realizeaza modificarea pachetului.....
    print("[**] Request handler")
    packet = IP(buffer) #decodific pachetul primit
    print(packet.summary())
    # de ex schimp ip -ul destinatie
    if IP in packet:
        packet[IP].dst = "8.8.8.8"
        print(f"[**] Destinatia a fost schimbata la {packet[IP].dst}")
    return bytes(packet)


# se modifica orice raspuns de la masina gazda/locala
def response_handler(buffer):
    # iar modific pachetul ....
```

```python
    print("[**] Response handler")
    packet = IP(buffer)
    # ex de modificare: adaug alta valoarea a TTL
    if IP in packet:
        packet[IP].ttl = 64
        print(f"[**] TTL modificat la {packet[IP].ttl}")
    return bytes(packet)
```

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ python3 proxy.py 127.0.0.1 9000 127.0.0.1 8080 True
[*] Ascult pe 127.0.0.1:9000
[==>] Primesc date de la 127.0.0.1:57732

[**] Response handler
[**] TTL modificat la 64
[<==] Trimit 20 octeti catre clientul local.
[==>] Am primit 78 octeti de la clientul local.
0000  47 45 54 20 2F 20 48 54 54 50 2F 31 2E 31 0D 0A   GET / HTTP/1.1..
0010  48 6F 73 74 3A 20 31 32 37 2E 30 2E 30 2E 31 3A   Host: 127.0.0.1:
0020  39 30 30 30 0D 0A 55 73 65 72 2D 41 67 65 6E 74   9000..User-Agent
0030  3A 20 63 75 72 6C 2F 37 2E 38 38 2E 31 0D 0A 41   : curl/7.88.1..A
0040  63 63 65 70 74 3A 20 2A 2F 2A 0D 0A 0D 0A         ccept: */*....
[**] Request handler
46.49.13.10 > 72.111.115.116 80 frag:2132 / Raw
[**] Destinatia a fost schimbata la 8.8.8.8
[==>] Trimit catre exterior
[<==] Am primit 457 octeti din afara
0000  48 54 54 50 2F 31 2E 30 20 32 30 30 20 4F 4B 0D   HTTP/1.0 200 OK.
0010  0A 53 65 72 76 65 72 3A 20 53 69 6D 70 6C 65 48   .Server: SimpleH
0020  54 54 50 2F 30 2E 36 20 50 79 74 68 6F 6E 2F 33   TTP/0.6 Python/3
0030  2E 31 31 2E 32 0D 0A 44 61 74 65 3A 20 46 72 69   .11.2..Date: Fri
0040  2C 20 31 33 20 44 65 63 20 32 30 32 34 20 31 32   , 13 Dec 2024 12
```

```
0010  0A 53 65 72 76 65 72 3A 20 53 69 6D 70 6C 65 48   .Server: SimpleH
0020  54 54 50 2F 30 2E 36 20 50 79 74 68 6F 6E 2F 33   TTP/0.6 Python/3
0030  2E 31 31 2E 32 0D 0A 44 61 74 65 3A 20 46 72 69   .11.2..Date: Fri
0040  2C 20 31 33 20 44 65 63 20 32 30 32 34 20 31 32   , 13 Dec 2024 12
0050  3A 30 35 3A 35 35 20 47 4D 54 0D 0A 43 6F 6E 74   :05:55 GMT..Cont
0060  65 6E 74 2D 74 79 70 65 3A 20 74 65 78 74 2F 68   ent-type: text/h
0070  74 6D 6C 3B 20 63 68 61 72 73 65 74 3D 75 74 66   tml; charset=utf
0080  2D 38 0D 0A 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67   -8..Content-Leng
0090  74 68 3A 20 33 30 32 0D 0A 0D 0A 3C 21 44 4F 43   th: 302....<!DOC
00A0  54 59 50 45 20 48 54 4D 4C 3E 0A 3C 68 74 6D 6C   TYPE HTML>.<html
00B0  20 6C 61 6E 67 3D 22 65 6E 22 3E 0A 3C 68 65 61    lang="en">.<hea
00C0  64 3E 0A 3C 6D 65 74 61 20 63 68 61 72 73 65 74   d>.<meta charset
00D0  3D 22 75 74 66 2D 38 22 3E 0A 3C 74 69 74 6C 65   ="utf-8">.<title
00E0  3E 44 69 72 65 63 74 6F 72 79 20 6C 69 73 74 69   >Directory listi
00F0  6E 67 20 66 6F 72 20 2F 3C 2F 74 69 74 6C 65 3E   ng for /</title>
0100  0A 3C 2F 68 65 61 64 3E 0A 3C 62 6F 64 79 3E 0A   .</head>.<body>.
0110  3C 68 31 3E 44 69 72 65 63 74 6F 72 79 20 6C 69   <h1>Directory li
0120  73 74 69 6E 67 20 66 6F 72 20 2F 3C 2F 68 31 3E   sting for /</h1>
0130  0A 3C 68 72 3E 0A 3C 75 6C 3E 0A 3C 6C 69 3E 3C   .<hr>.<ul>.<li><
0140  61 20 68 72 65 66 3D 22 2E 69 64 65 61 2F 22 3E   a href=".idea/">
0150  2E 69 64 65 61 2F 3C 2F 61 3E 3C 2F 6C 69 3E 0A   .idea/</a></li>.
0160  3C 6C 69 3E 3C 61 20 68 72 65 66 3D 22 2E 76 65   <li><a href=".ve
0170  6E 76 2F 22 3E 2E 76 65 6E 76 2F 3C 2F 61 3E 3C   nv/">.venv/</a><
0180  2F 6C 69 3E 0A 3C 6C 69 3E 3C 61 20 68 72 65 66   /li>.<li><a href
0190  3D 22 70 72 6F 78 79 2E 70 79 22 3E 70 72 6F 78   ="proxy.py">prox
01A0  79 2E 70 79 3C 2F 61 3E 3C 2F 6C 69 3E 0A 3C 2F   y.py</a></li>.</
01B0  75 6C 3E 0A 3C 68 72 3E 0A 3C 2F 62 6F 64 79 3E   ul>.<hr>.</body>
01C0  0A 3C 2F 68 74 6D 6C 3E 0A                        .</html>.
[**] Response handler
[**] TTL modificat la 64
[<==] Trimis local.
[*] Nu mai sunt date, inchid conexiunea.
```

3) **PING:**

a) **Python:** am testat toate functiile date de program:

```python
if __name__ == '__main__':
    verbose_ping('google.com')
    verbose_ping('127.0.0.1')

    print('Testare do_one_ping pt google.com:')
    delay = do_one_ping('google.com')
    if delay is None:
        print("Nu a raspuns la ping")
    else:
        print(f"Raspuns primit in {delay * 1000} ms")

    print('\nTestare do_one_ping pt 127.0.0.1:')
    delay = do_one_ping('127.0.0.1')
    if delay is None:
        print("Nu a raspuns la ping")
    else:
        print(f"Raspuns primit in {delay * 1000} ms")

    print('\nTestare multi_ping_query pt google.com:')
    delays = multi_ping_query( dest_addr: 'google.com', count=5)
    for i, delay in enumerate(delays, 1):
        if delay is None:
            print(f"Ping {i}: Nu a raspuns")
        else:
            print(f"Ping {i}: Raspuns in {delay} ms")

    print('\nTestare PingQuery pt google.com:')
    ping_query = PingQuery( host: 'google.com', p_id=1)
    ping_query.handle_connect()
    ping_query.handle_write()
    ping_query.handle_read()

    if ping_query.get_result() is not None:
        print(f"Raspuns primit de la {ping_query.get_host()} in {ping_query.get_result() * 1000} ms")
    else:
        print("Nu a raspuns")
```

```
root@debian:/home/diana/PycharmProjects/Lab11# python3 ex2.py
/home/diana/PycharmProjects/Lab11/ex2.py:6: DeprecationWarning: The asyncore module is deprecated and will be removed in Python 3.12. The recommended replacement is asyncio
  import asyncore
ping google.com...
am primit raspuns in 35.9128 milliseconds.

ping google.com...
am primit raspuns in 34.8804 milliseconds.

ping google.com...
am primit raspuns in 36.4504 milliseconds.

ping google.com...
am primit raspuns in 35.3947 milliseconds.

ping 127.0.0.1...
am primit raspuns in 0.015 milliseconds.

ping 127.0.0.1...
am primit raspuns in 0.0153 milliseconds.

ping 127.0.0.1...
am primit raspuns in 0.0076 milliseconds.

ping 127.0.0.1...
am primit raspuns in 0.0098 milliseconds.

Testare do_one_ping pt google.com:
Raspuns primit in 34.12747383117676 ms

Testare do_one_ping pt 127.0.0.1:
Raspuns primit in 0.010251998901367188 ms
```

```
ping 127.0.0.1...
am primit raspuns in 0.0098 milliseconds.

Testare do_one_ping pt google.com:
Raspuns primit in 34.12747383117676 ms

Testare do_one_ping pt 127.0.0.1:
Raspuns primit in 0.010251998901367188 ms

Testare multi_ping_query pt google.com:
Ping 1: Raspuns in 37.8366 ms
Ping 2: Raspuns in 34.1375 ms
Ping 3: Raspuns in 34.198 ms
Ping 4: Raspuns in 34.4462 ms
Ping 5: Raspuns in 37.0607 ms

Testare PingQuery pt google.com:
Raspuns primit de la google.com in 6.803750991821289 ms
root@debian:/home/diana/PycharmProjects/Lab11#
```

b) **C#:**

```
root@debian:/home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex2/Lab11_Ex2# dotnet run
ping google.com...
am primit raspuns in 36.6154 milliseconds.

ping google.com...
(eroare nu a raspuns dupa 2 secunde.)

ping google.com...
am primit raspuns in 32.6846 milliseconds.

ping google.com...
(eroare nu a raspuns dupa 2 secunde.)

ping 127.0.0.1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)

ping 127.0.0.1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)

ping 127.0.0.1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)

ping 127.0.0.1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)

Testare do_one_ping pt google.com:
Raspuns primit in 42.1841 ms

Testare do_one_ping pt 127.0.0.1:
Eroare la deschiderea socketului: Address family not supported by protocol
Nu a raspuns la ping
```

```
ping 127.0.0.1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)

Testare do_one_ping pt google.com:
Raspuns primit in 42.1841 ms

Testare do_one_ping pt 127.0.0.1:
Eroare la deschiderea socketului: Address family not supported by protocol
Nu a raspuns la ping

Testare multi_ping_query pt google.com:
Ping 1: Nu a raspuns
Ping 2: Raspuns in 36.1818 ms
Ping 3: Raspuns in 35.5303 ms
Ping 4: Raspuns in 36.1188 ms
Ping 5: Nu a raspuns

Testare PingQuery pt google.com:
Nu a raspuns
```

Observatii: În general în python se aleg protocoalele în mod automat în funcție de adresa disponibilă. În schimb, C# este mai strict și necesită o gestionare explicită a protocolului utilizat. Dacă încerc sa schimb în InterNetworkV6 (ca sa functioneze cu IPV6 deoarece nu se convertește automat la rețea în C#, atunci nu mai merge niciun ping nici pe google). Am încercat sa mai adaug niște comentarii ca să depistez problema, însă problema principală este faptul că nu recunoaște protocolul.

```
ping 127.0.0.1...
Rezolv hostul 127.0.0.1...
Adresa detectata: ::1 (Familia: InterNetworkV6)
Adresa detectata: 127.0.0.1 (Familia: InterNetwork)
Pachet ICMP creat (lungime: 200 bytes, checksum: 5F01).
Trimit pachet ICMP catre ::1...
Eroare la deschiderea socketului: Address family not supported by protocol
(eroare nu a raspuns dupa 2 secunde.)
```

C/C++

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
using System.Linq;

class Program
{
    private const int ICMP_ECHO_REQUEST = 8;

    static readonly string[] ERROR_DESCR = {
        "-icmp numai ca root lansati din terminalul proiectului programul cu
sudo dotnet build si dotnet run",
        "-icmp poate fi transmis numai de utilizatori sau procese cu drept de
root"
    };

    static ushort checksum(byte[] source)
    {
        uint sum = 0;
        int countTo = (source.Length / 2) * 2;
        int count = 0;
        while (count < countTo)
        {
            uint thisVal = (uint)(source[count + 1] * 256 + source[count]);
            sum = sum + thisVal;
            sum = sum & 0xffffffff;
            count += 2;
        }
        if (countTo < source.Length)
        {
            sum = sum + source[source.Length - 1];
            sum = sum & 0xffffffff;
        }
        sum = (sum >> 16) + (sum & 0xffff);
        sum = sum + (sum >> 16);
        uint answer = ~sum;
        answer = answer & 0xffff;
        answer = (answer >> 8) | ((answer << 8) & 0xff00);
        return (ushort)answer;
    }

    static byte[] create_packet(int id)
```

```
    {
        byte[] header = new byte[8];
        header[0] = ICMP_ECHO_REQUEST;  // type
        header[1] = 0;  // code
        header[2] = 0;  // checksum placeholder
        header[3] = 0;  // -//-
        header[4] = (byte)(id >> 8);  // ID high byte
        header[5] = (byte)(id & 0xFF);  // ID low byte
        header[6] = 1;  // Sequence number
        header[7] = 0;  // Sequence number (poate varia)

        string data = new string('Q', 192);
        byte[] dataBytes = Encoding.ASCII.GetBytes(data);
        byte[] packet = new byte[header.Length + dataBytes.Length];
        Array.Copy(header, 0, packet, 0, header.Length);
        Array.Copy(dataBytes, 0, packet, header.Length, dataBytes.Length);

        ushort checksumValue = checksum(packet);
        packet[2] = (byte)(checksumValue >> 8);
        packet[3] = (byte)(checksumValue & 0xFF);

        return packet;
    }

    static double? do_one_ping(string destAddr, int timeout = 1)
    {
        try
        {
            using (Socket mySocket = new Socket(AddressFamily.InterNetwork,
SocketType.Raw, (ProtocolType)1)) // ICMP = 1
            {
                IPAddress host = Dns.GetHostEntry(destAddr).AddressList[0];
                int packetId = (int)((Guid.NewGuid().GetHashCode() * new
Random().NextDouble()) % 65535);
                byte[] packet = create_packet(packetId);

                mySocket.SendTo(packet, new IPEndPoint(host, 0));

                return receive_ping(mySocket, packetId, DateTime.Now, timeout);
            }
        }
        catch (SocketException e)
        {
```

```
            Console.WriteLine("Eroare la deschiderea socketului: " +
e.Message);
            return null;
        }
    }

    static double? receive_ping(Socket mySocket, int packetId, DateTime
timeSent, int timeout)
    {
        DateTime timeoutLimit = timeSent.AddSeconds(timeout);
        while (DateTime.Now < timeoutLimit)
        {
            if (mySocket.Available > 0)
            {
                byte[] buffer = new byte[1024];
                EndPoint remoteEndPoint = new IPEndPoint(IPAddress.Any, 0);
                int bytesRead = mySocket.ReceiveFrom(buffer, ref
remoteEndPoint);
                byte[] icmpHeader = buffer.Skip(20).Take(8).ToArray();
                byte type = icmpHeader[0];
                byte code = icmpHeader[1];
                ushort checksum = (ushort)((icmpHeader[2] << 8) |
icmpHeader[3]);
                int pId = (icmpHeader[4] << 8) | icmpHeader[5];

                if (pId == packetId)
                {
                    return (DateTime.Now - timeSent).TotalSeconds;
                }
            }
        }
        return null;
    }

    static void verbose_ping(string destAddr, int timeout = 2, int count = 4)
    {
        for (int i = 0; i < count; i++)
        {
            Console.WriteLine($"ping {destAddr}...");
            double? delay = do_one_ping(destAddr, timeout);
            if (delay == null)
            {
                Console.WriteLine($"(eroare nu a raspuns dupa {timeout}
secunde.)");
```

```
            }
            else
            {
                Console.WriteLine($"am primit raspuns in
{Math.Round(delay.Value * 1000, 4)} milliseconds.");
            }
            Console.WriteLine();
        }
    }

    static double[] multi_ping_query(string destAddr, int timeout = 1, int
count = 4)
    {
        double[] delays = new double[count];
        for (int i = 0; i < count; i++)
        {
            double? delay = do_one_ping(destAddr, timeout);
            delays[i] = delay.HasValue ? Math.Round(delay.Value * 1000, 4) :
-1;  // -1 inseamna niciun raspuns
        }
        return delays;
    }

    class PingQuery
    {
        private Socket socket;
        private DateTime timeReceived;
        private DateTime timeSent;
        private int timeout;
        private int packetId;
        private string host;
        private byte[] packet;

        public PingQuery(string host, int pId, int timeout = 500)
        {
            this.socket = new Socket(AddressFamily.InterNetwork,
SocketType.Raw, (ProtocolType)1);
            this.timeReceived = DateTime.MinValue;
            this.timeSent = DateTime.MinValue;
            this.timeout = timeout;
            this.packetId = (int)((Guid.NewGuid().GetHashCode() / pId) %
65535);
            this.host = host;
            this.packet = create_packet(this.packetId);
```

```
        }

        public void handle_connect() { }
        public void handle_write()
        {
            timeSent = DateTime.Now;
            socket.SendTo(packet, new
IPEndPoint(Dns.GetHostEntry(host).AddressList[0], 0));
        }

        public void handle_read()
        {
            DateTime readTime = DateTime.Now;
            if (socket.Available > 0)
            {
                byte[] buffer = new byte[1024];
                EndPoint remoteEndPoint = new IPEndPoint(IPAddress.Any, 0);
                int bytesRead = socket.ReceiveFrom(buffer, ref remoteEndPoint);
                byte[] icmpHeader = buffer.Skip(20).Take(8).ToArray();
                byte type = icmpHeader[0];
                byte code = icmpHeader[1];
                ushort checksum = (ushort)((icmpHeader[2] << 8) |
icmpHeader[3]);
                int pId = (icmpHeader[4] << 8) | icmpHeader[5];

                if (pId == packetId)
                {
                    timeReceived = readTime;
                }
            }
        }

        public double? get_result()
        {
            if (timeReceived > DateTime.MinValue)
            {
                return (timeReceived - timeSent).TotalSeconds;
            }
            return null;
        }

        public string get_host() => host;
    }
```

```
static void Main()
{
    verbose_ping("google.com");
    verbose_ping("127.0.0.1");

    Console.WriteLine("Testare do_one_ping pt google.com:");
    double? delay = do_one_ping("google.com");
    if (delay == null)
    {
        Console.WriteLine("Nu a raspuns la ping");
    }
    else
    {
        Console.WriteLine($"Raspuns primit in {delay * 1000} ms");
    }

    Console.WriteLine("\nTestare do_one_ping pt 127.0.0.1:");
    delay = do_one_ping("127.0.0.1");
    if (delay == null)
    {
        Console.WriteLine("Nu a raspuns la ping");
    }
    else
    {
        Console.WriteLine($"Raspuns primit in {delay * 1000} ms");
    }

    Console.WriteLine("\nTestare multi_ping_query pt google.com:");
    double[] delays = multi_ping_query("google.com", count: 5);
    for (int i = 0; i < delays.Length; i++)
    {
        if (delays[i] == -1)
        {
            Console.WriteLine($"Ping {i + 1}: Nu a raspuns");
        }
        else
        {
            Console.WriteLine($"Ping {i + 1}: Raspuns in {delays[i]} ms");
        }
    }

    Console.WriteLine("\nTestare PingQuery pt google.com:");
    PingQuery pingQuery = new PingQuery("google.com", 1);
    pingQuery.handle_connect();
```

```
        pingQuery.handle_write();
        pingQuery.handle_read();

        double? result = pingQuery.get_result();
        if (result.HasValue)
        {
            Console.WriteLine($"Raspuns primit de la {pingQuery.get_host()} in
{result * 1000} ms");
        }
        else
        {
            Console.WriteLine("Nu a raspuns");
        }
    }
}
```

4) Interceptare de pachete:

a) **Python:**

```
root@debian:/home/diana/PycharmProjects/Lab11# python3 ex3.py
*******************TCPv4***********************
Versiune:4
lungimeheader:20
TTL:64
protocol:6
Sursa:127.0.0.1
Destinatia:127.0.0.1
*****SegmentulTCP*****
Portul sursei:63343
Destination Port:41334
Secventa:3952928012
Avizare:1008185712
*****Flag-uri*****
URG:0
ACK:0
PSH:0
RST:0
SYN:0
FIN:0
*****dateTCP*****
\x31.\x0d.\x0a.\x20.\x0d.\x0a
*******************TCPv4***********************
Versiune:4
lungimeheader:20
TTL:64
protocol:6
Sursa:127.0.0.1
```

```
Portul sursei:63343
Destination Port:41334
Secventa:3952928012
Avizare:1008185712
*****Flag-uri*****
URG:0
ACK:0
PSH:0
RST:0
SYN:0
FIN:0
*****dateTCP*****
\x31.\x0d.\x0a.\x20.\x0d.\x0a
*******************TCPv4************************
Versiune:4
lungimeheader:20
TTL:64
protocol:6
Sursa:127.0.0.1
Destinatia:127.0.0.1
*****SegmentulTCP*****
Portul sursei:41334
Destination Port:63343
Secventa:1008185712
Avizare:3952928018
*****Flag-uri*****
URG:0
ACK:0
```

b) C#: nu reușește să capteze niciun pachet

```
Lab11_Ex3 succeeded with 8 warning(s) (0.8s) → Lab11_Ex3/bin/Debug/net9.0/Lab11_Ex3.dll
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(10,19): warning CS0414: The field 'Program.TAB_1' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(11,19): warning CS0414: The field 'Program.TAB_2' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(12,19): warning CS0414: The field 'Program.TAB_3' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(14,19): warning CS0414: The field 'Program.DATA_TAB_1' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(13,19): warning CS0414: The field 'Program.TAB_4' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(17,19): warning CS0414: The field 'Program.DATA_TAB_4' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(16,19): warning CS0414: The field 'Program.DATA_TAB_3' is assigned but its value is never used
  /home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3/Program.cs(15,19): warning CS0414: The field 'Program.DATA_TAB_2' is assigned but its value is never used

Build succeeded with 8 warning(s) in 2.9s
root@debian:/home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3# cd Lab11_Ex3
root@debian:/home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3# dotnet run
^Croot@debian:/home/diana/Documents/SSC/ProgrameRescrise/Lab11_Ex3/Lab11_Ex3# dotnet run
```

c) **Ruby: - la fel merge fără eroare însă nu capteaza nimic:**

```
        from capture_packets.rb:219:in `<main>'
root@debian:/home/diana/Ruby# nano capture_packets.rb
root@debian:/home/diana/Ruby# ruby capture_packets.rb
```

Unset

```
require 'socket'
require 'ipaddr'
require 'stringio'

TAB_1 = "\t-"
TAB_2 = "\t\t-"
TAB_3 = "\t\t\t-"
TAB_4 = "\t\t\t\t-"
DATA_TAB_1 = "\t "
DATA_TAB_2 = "\t\t "
DATA_TAB_3 = "\t\t\t "
DATA_TAB_4 = "\t\t\t\t "

# despachetare cadru Ethernet
def ethernet_frame(data)
  dst_mac, src_mac, proto_type = data[0, 6], data[6, 6], data[12,
2].unpack('n')[0]
  proto = ''
  if proto_type == 0x0800
    proto = 'IPV4'
  elsif proto_type == 0x86dd
    proto = 'IPV6'
  end
  data = data[14..-1]
  [dst_mac, src_mac, proto, data]
end

def get_mac_addr(bytes_addr)
  bytes_addr.map { |byte| byte.to_s(16).rjust(2, '0') }.join(':').upcase
end

def ipv4(addr)
  addr.join('.')
end

# formatarea liniei de iesire
```

```
def format_output_line(prefix, string)
  size = 80 - prefix.length
  string = string.each_byte.map { |byte| '\\x' + byte.to_s(16).rjust(2, '0')
}.join('.') if string.is_a?(String)
  size -= 1 if size.odd?
  string.scan(/.{1,#{size}}/).join("\n").prepend(prefix)
end

def tcp_seg(data)
  src_port, dest_port, sequence, acknowledgement, offset_reserved_flag =
data[0, 14].unpack('n2N2n')
  offset = (offset_reserved_flag >> 12) * 4
  flags = {
    urg: (offset_reserved_flag & 0x20) >> 5,
    ack: (offset_reserved_flag & 0x10) >> 4,
    psh: (offset_reserved_flag & 0x08) >> 3,
    rst: (offset_reserved_flag & 0x04) >> 2,
    syn: (offset_reserved_flag & 0x02) >> 1,
    fin: offset_reserved_flag & 0x01
  }
  [src_port, dest_port, sequence, acknowledgement, flags, data[offset..-1]]
end

def udp_seg(data)
  src_port, dest_port, size = data[0, 8].unpack('n3')
  [src_port, dest_port, size, data[8..-1]]
end

def icmp_packet(data)
  icmp_type, code, checksum = data[0, 4].unpack('CCn')
  [icmp_type, code, checksum, data[4..-1]]
end

def icmpv6Header(data)
  ipv6_icmp_type, ipv6_icmp_code, ipv6_icmp_chekcsum = data[0, 4].unpack('CCn')
  puts "*******************ICMPv6***********************"
  puts "\tTip ICMPv6: #{ipv6_icmp_type}"
  puts "\tCod ICMPv6: #{ipv6_icmp_code}"
  puts "\tSuma de control ICMPv6: #{ipv6_icmp_chekcsum}"
  data[4..-1]
end

def ipv6Header(data, filter)
```

```
  ipv6_first_word, ipv6_payload_length, ipv6_next_header, ipv6_hop_limit =
data[0, 8].unpack('NnCC')
  ipv6_src_ip = IPAddr.ntop(data[8, 16])
  ipv6_dst_ip = IPAddr.ntop(data[24, 16])
  version = ipv6_first_word >> 28
  traffic_class = (ipv6_first_word >> 16) & 0x0FFF
  flow_label = ipv6_first_word & 0xFFFF
  ipv6_next_header = nextHeader(ipv6_next_header)
  [data[40..-1], ipv6_next_header]
end

def nextHeader(ipv6_next_header)
  case ipv6_next_header
  when 6 then 'TCP'
  when 17 then 'UDP'
  when 43 then 'Rutare'
  when 1 then 'ICMP'
  when 58 then 'ICMPv6'
  when 44 then 'Fragment'
  when 0 then 'HOPOPT'
  when 60 then 'Destinatie'
  when 51 then 'Autentificare'
  when 50 then 'Incapsulare'
  else 'Unknown'
  end
end

def printPacketsV4(filter, data, raw_data)
  version, header_length, ttl, proto, src, target, data = ipv4_Packet(data)
  if proto == 1 && (filter.empty? || filter[1] == 1)
    icmp_type, code, checksum, data = icmp_packet(data)
    puts "*******************ICMP***********************"
    puts "\ttip ICMP: #{icmp_type}"
    puts "\tcod ICMP: #{code}"
    puts "\tsumacontrol ICMP: #{checksum}"
  elsif proto == 6 && (filter.empty? || filter[1] == 6)
    puts "*******************TCPv4***********************"
    puts "Versiune: #{version}\nlungimeheader: #{header_length}\nTTL: #{ttl}"
    puts "protocol: #{proto}\nSursa: #{src}\nDestinatia: #{target}"
    src_port, dest_port, sequence, acknowledgment, flags, data = tcp_seg(data)
    puts "*****SegmentulTCP*****"
    puts "Portul sursei: #{src_port}\nDestination Port: #{dest_port}"
    puts "Secventa: #{sequence}\nAvizare: #{acknowledgment}"
    puts "*****Flag-uri*****"
```

```
    flags.each { |key, value| puts "#{key.upcase}: #{value}" }
    if !data.empty?
      if src_port == 80 || dest_port == 80
        puts '*****dateHTTP*****'
        begin
          http_info = data.split("\n")
          http_info.each { |line| puts line }
        rescue => e
          puts "Eroare la parsarea HTTP: #{e.message}"
          puts format_output_line("", data)
        end
      else
        puts '*****dateTCP*****'
        puts format_output_line("", data)
      end
    end
  elsif proto == 17 && (filter.empty? || filter[1] == 17)
    puts "******************UDPv4***********************"
    puts "Versiune: #{version}\nLungime header: #{header_length}\nTTL: #{ttl}"
    puts "protocol: #{proto}\nSursa: #{src}\nDestinatia: #{target}"
    src_port, dest_port, length, data = udp_seg(data)
    puts "*****Segmentul UDP*****"
    puts "Portul Sursei: #{src_port}\nPortul destinatiei: #{dest_port}\nLength:
#{length}"
  end
end

def printPacketsV6(filter, nextProto, newPacket)
  remainingPacket = ""
  if nextProto == 'ICMPv6' && (filter.empty? || filter[2] == "ICMPv6")
    remainingPacket = icmpv6Header(newPacket)
  elsif nextProto == 'TCP' && (filter.empty? || filter[2] == "TCP")
    remainingPacket = tcpHeader(newPacket)
  elsif nextProto == 'UDP' && (filter.empty? || filter[2] == "UDP")
    remainingPacket = udpHeader(newPacket)
  end
  remainingPacket
end

def tcpHeader(newPacket)
  packet = newPacket.unpack('n2N2n4')
  src_port, dst_port, sqnc_num, ackn_num, data_offset, tcp_flags, window,
checksum, urg_ptr = packet
  flags = {
```

```
    urg: tcp_flags & 0x0020,
    ack: tcp_flags & 0x0010,
    push: tcp_flags & 0x0008,
    reset: tcp_flags & 0x0004,
    syn: tcp_flags & 0x0002,
    fin: tcp_flags & 0x0001
  }
  puts "*******************TCP***********************"
  puts "\tPortul sursei: #{src_port}"
  puts "\tPortul destinatiei: #{dst_port}"
  puts "\tNumarul secventei: #{sqnc_num}"
  puts "\tNumarul validarii: #{ackn_num}"
  puts "\tData Offset: #{data_offset}"
  puts "\tFlag-uri TCP: #{tcp_flags}"
  flags.each { |key, value| puts "#{key.upcase}: #{value == 0 ? 'Inactiv' :
'Activ'}" }
  puts "\tFereastra: #{window}"
  puts "\tSuma de control: #{checksum}"
  puts "\tUrgent Pointer: #{urg_ptr}"
  newPacket[20..-1]
end

def udpHeader(newPacket)
  packet = newPacket.unpack('n3')
  src_port, dst_port, length, checksum = packet
  puts "*******************UDP***********************"
  puts "\tPortul sursei: #{src_port}"
  puts "\tPortul destinatiei: #{dst_port}"
  puts "\tLungime: #{length}"
  puts "\tSuma de control: #{checksum}"
  newPacket[8..-1]
end

def main
  conn = Socket.new(Socket::AF_PACKET, Socket::SOCK_RAW, 0x0003)  # 0x0003 is
ETH_P_ALL
  filters = [['ICMP', 1, 'ICMPv6'], ['UDP', 17, 'UDP'], ['TCP', 6, 'TCP']]
  filter = []
  if ARGV.length == 1
    puts "acesta este filtrul: #{ARGV[0]}"
    filters.each do |f|
      if ARGV[0] == f[0]
        filter = f
      end
```

```
    end
  end
  while true
    raw_data, addr = conn.recvfrom(65536)
    dst_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
    if eth_proto == 'IPV6'
      new_packet, next_proto = ipv6Header(data, filter)
      printPacketsV6(filter, next_proto, new_packet)
    elsif eth_proto == 'IPV4'
      printPacketsV4(filter, data, raw_data)
    end
  end
end

main
```

Tema

Dacă ați înțeles aceste programe de acum pe baza lor puteți face orice fel de combinații doriți. De exemplu un proxy care face analiza pachetelor primite și eventual le trimite modificate mai departe sau chiar nu le trimite. Dacă mai săpați o să vedeți că o serie de abordări moderne din programare care sunt declarate sigure sunt extrem de sensibile la astfel de analize! Limbaje în care se poate lucra pentru temă - golang,rust,ruby,cpp,c,java,.net

5)

a) **Python=>** fluxul de lucru**:**
   i) Clientul trimite o cerere HTTP (de exemplu, de tip GET) către proxy pe portul 8888.
   ii) Proxy-ul interceptează cererea, o analizează și, dacă este necesar, o modifică (ex. schimbă User-Agent).
   iii) Proxy-ul trimite cererea modificată sau originală către serverul de destinație (pe portul 80).
   iv) Serverul răspunde cu un mesaj de succes și un text ("Hello from the server!").
   v) Proxy-ul primește răspunsul, îl trimite înapoi către client și închide conexiunea.

server.py

```python
import socket
```

```python
HOST = '127.0.0.1'
PORT = 80


def handle_client(client_socket):
    try:
        request = client_socket.recv(1024)
        print(f"Received request: {request.decode()}")

        response = "HTTP/1.1 200 OK\r\n"
        response += "Content-Type: text/plain\r\n"
        response += "Connection: close\r\n\r\n"
        response += "Hello from the server!"

        client_socket.send(response.encode())
    except Exception as e:
        print(f"Error handling client request: {e}")
    finally:
        client_socket.close()

def run_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((HOST, PORT))
    server_socket.listen(5)
    print(f"[*] Server listening on {HOST}:{PORT}...")

    while True:
        client_socket, addr = server_socket.accept()
        print(f"[*] Accepted connection from {addr}")
        handle_client(client_socket)


if __name__ == "__main__":
    run_server()
```

```
[*] Accepted connection from ('127.0.0.1', 34772)
Received request: GET / HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Connection: close
```

client.py

```python
import socket

PROXY_HOST = '127.0.0.1'
PROXY_PORT = 8888

def create_http_request():
   #cerere http simpla
   request = "GET / HTTP/1.1\r\n"
   request += "Host: example.com\r\n"
   request += "User-Agent: Mozilla/5.0\r\n"  # User-Agent standard
   request += "Connection: close\r\n\r\n"
   return request.encode()
def run_client():
    try:
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client_socket.settimeout(5)
        client_socket.connect((PROXY_HOST, PROXY_PORT))

        request = create_http_request()
        print("Sending request to proxy...")
        client_socket.send(request)

        response = client_socket.recv(1024)
        print("Received response from server:")
        print(response.decode())

    except socket.timeout:
        print("Connection timed out")
    except Exception as e:
        print(f"Error: {e}")
    finally:
        client_socket.close()
```

```python
if __name__ == "__main__":
    run_client()
```

```
root@debian:/home/diana/PycharmProjects/Lab11# python3 client.py
Sending request to proxy...
Received response from server:
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: close
```

proxy.py

```python
import socket
import time

import scapy.all as scapy
from threading import Thread
from scapy.layers.inet import IP, ICMP,  TCP
from scapy.all import Raw
import logging
# setari proxy
PROXY_HOST = '127.0.0.1'  # adr de la care proxy ul va asculta
PROXY_PORT = 8888  # portul la care va asculta
FORWARD_HOST = '127.0.0.1'  # ip ul server ului la care vor fi trimise
pachetele
FORWARD_PORT = 80  # portul serverului

#adaug si logging
logging.basicConfig(filename="proxy_log.txt", level=logging.INFO)
def log_packet(packet, direction="Inbound"):
    timestamp = time.strftime('%Y-%m-%d %H:%M:%S', time.gmtime())
    logging.info(f"{direction} Packet at {timestamp}: {packet.summary()}")
    logging.info(f"Raw Data: {packet.show()}")

def modify_packet(packet):
    print(f"Received packet: {packet.summary()}")
    # 1. blochez pachetele icmp
    if packet.haslayer(ICMP):
        print("ICMP packet detected, blocking it!")
```

```
        return None  # nu trimit mai departe

    # 2.modific head url http daca este un pachet de acest tip
    if packet.haslayer(TCP):
        if packet.haslayer(Raw):
            raw_data = packet.getlayer(Raw).load.decode('utf-8', 'ignore')

            # verific daca este un header http prin cautarea a get, post etc
            if "GET" in raw_data or "POST" in raw_data:
                print("Modifying HTTP request...")

                # modific user agent ul
                modified_data = raw_data.replace("User-Agent: Mozilla",
"User-Agent: CustomProxy")

                # inlocuiesc datele brute cu cele modif
                packet.getlayer(Raw).load = modified_data.encode('utf-8')
                print(f"Modified HTTP request: {modified_data}")
    return packet
def handle_client(client_socket):
    request = client_socket.recv(1024)

    # analizez pachetul primit
    packet = IP(request)

    print(f"Received packet: {packet.summary()}")
    modified_packet = modify_packet(packet)
    if modified_packet is None:
        # daca nu avem pachet inchid conex
        client_socket.close()
        return

    # trimit pachetul catre server ul destinatie
    forward_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    forward_socket.connect((FORWARD_HOST, FORWARD_PORT))
    forward_socket.send(request)

    # astept raspuns
    response = forward_socket.recv(1024)
    log_packet(IP(response), "Outbound")
    print(f"Received response: {response}")

    # trimit raspunsul inapoi la client
    client_socket.send(response)
```

```python
    client_socket.close()


def start_proxy():
    # creez un socket sa ascult conexiunile
    proxy = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    proxy.bind((PROXY_HOST, PROXY_PORT))
    proxy.listen(5)
    print(f"[*] Listening on {PROXY_HOST}:{PROXY_PORT}...")

    while True:
        client_socket, addr = proxy.accept()
        print(f"[*] Accepted connection from {addr}")

        # thread pentru a gestiona conex
        client_handler = Thread(target=handle_client, args=(client_socket,))
        client_handler.start()


if __name__ == "__main__":
    start_proxy()
```

```
(.venv) diana@debian:~/PycharmProjects/Lab11$ python3 tema.py
[*] Listening on 127.0.0.1:8888...
[*] Accepted connection from ('127.0.0.1', 56330)
Received packet: 46.49.13.10 > 72.111.115.116 80 frag:2132 / Raw
Received packet: 46.49.13.10 > 72.111.115.116 80 frag:2132 / Raw
###[ IP ]###
  version   = 4
  ihl       = 8
  tos       = 0x54
  len       = 21584
  id        = 12081
  flags     = MF
  frag      = 3633
  ttl       = 32
  proto     = esp
  chksum    = 0x3030
  src       = 32.79.75.13
  dst       = 10.67.111.110
  \options   \
   |###[ IP Option Router Alert ]###
   |  copy_flag = 0
   |  optclass  = 3
   |  option    = router_alert
   |  length    = 101
   |  alert     = 28276
   |###[ IP Option ]###
   |  copy_flag = 0
   |  optclass  = 1
   |  option    = flow_control
   |  length    = 84
   |  value     = b'ype: t'
###[ ESP ]###
     spi       = 0x6578742f
     seq       = 1886151017
     data      = 6e0d0a436f6e6e656374696f6e3a20636c6f73650d0a0d0a48656c6c6f2066726f6d207468652073657276657221

Received response: b'HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nConnection: close\r\n\r\nHello from the server!'
```

Probleme: Se fragmenteaza pachetul și datorită fragmentării nu se schimbă corespunzător User-Agent-ul. În Ruby nu se fragmentează pachetul astfel se modifică corespunzător pachetul.

b) Ruby:

server.rb

```
Unset
require 'socket'

HOST = '127.0.0.1'
PORT = 80

def handle_client(client_socket)
  begin
```

```
    # primesc cererea de la client
    request = client_socket.recv(1024)
    puts "Received request: #{request}"

    # creez raspunsul
    response = "HTTP/1.1 200 OK\r\n"
    response += "Content-Type: text/plain\r\n"
    response += "Connection: close\r\n\r\n"
    response += "Hello from the server!"

    # trimit raspunsul inapoi la client
    client_socket.write(response)
    client_socket.close
  rescue => e
    puts "Error handling client request: #{e}"
    client_socket.close
  end
end

def run_server
  # creez un socket sa ascult conexiunile
  server_socket = TCPServer.new(HOST, PORT)
  puts "[*] Server listening on #{HOST}:#{PORT}..."

  while true
    # accept conexiunile
    client_socket = server_socket.accept
    addr = client_socket.peeraddr
    puts "[*] Accepted connection from #{addr[3]}:#{addr[1]}"
    handle_client(client_socket)
  end
end

if __FILE__ == $0
  run_server
end
```

```
root@debian:/home/diana/Ruby# ruby server.rb
[*] Server listening on 127.0.0.1:80...
[*] Accepted connection from 127.0.0.1:33190
Received request: GET / HTTP/1.1
Host: example.com
User-Agent: CustomProxy/5.0
Connection: close
```

client.rb

```
Unset
require 'socket'

PROXY_HOST = '127.0.0.1'
PROXY_PORT = 8888

# creez o cerere HTTP simpla
def create_http_request
  request = "GET / HTTP/1.1\r\n"
  request += "Host: example.com\r\n"
  request += "User-Agent: Mozilla/5.0\r\n" # User-Agent standard
  request += "Connection: close\r\n\r\n"
  request
end

def run_client
  begin
    # creez socket-ul clientului
    client_socket = TCPSocket.new(PROXY_HOST, PROXY_PORT)

    # trimit cererea catre proxy
    request = create_http_request
    puts "Sending request to proxy..."
    client_socket.write(request)

    # primesc raspunsul de la server
    response = client_socket.recv(1024)
    puts "Received response from server:"
    puts response

    client_socket.close
  rescue => e
    puts "Error: #{e}"
```

```
  end
end

if __FILE__ == $0
  run_client
end
```

```
root@debian:/home/diana/Ruby# ruby client.rb
Sending request to proxy...
Received response from server:
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: close

Hello from the server!
root@debian:/home/diana/Ruby#
```

proxy.rb

```
Unset
require 'socket'
require 'thread'

# setari proxy
PROXY_HOST = '127.0.0.1'  # adr de la care proxy ul va asculta
PROXY_PORT = 8888         # portul la care va asculta
FORWARD_HOST = '127.0.0.1' # ip ul server ului la care vor fi trimise pachetele
FORWARD_PORT = 80          # portul serverului

# modific headerul http daca este un pachet de acest tip
def modify_request(request)
  puts "Received request:\n#{request}"

  # verific daca este un header http prin cautarea a GET, POST etc
  if request.include?("GET") || request.include?("POST")
    puts "Modifying HTTP request..."

    # modific User-Agent-ul
```

```
    modified_request = request.gsub("User-Agent: Mozilla", "User-Agent:
CustomProxy")

    # inlocuiesc datele brute cu cele modificate
    puts "Modified HTTP request:\n#{modified_request}"
    return modified_request
  end

  request
end

def handle_client(client_socket)
  begin
    # primesc cererea de la client
    request = client_socket.recv(1024)

    # analizez pachetul primit
    puts "Received request from client:\n#{request}"

    # modific pachetul daca este cazul
    modified_request = modify_request(request)

    # trimit pachetul catre server-ul destinatie
    server_socket = TCPSocket.new(FORWARD_HOST, FORWARD_PORT)
    server_socket.write(modified_request)

    # astept raspuns
    response = server_socket.recv(1024)
    puts "Received response from server:\n#{response}"

    # trimit raspunsul inapoi la client
    client_socket.write(response)

    server_socket.close
    client_socket.close
  rescue => e
    puts "Error handling client request: #{e}"
    client_socket.close
  end
end

def start_proxy
  # creez un socket sa ascult conexiunile
  proxy = TCPServer.new(PROXY_HOST, PROXY_PORT)
```

```
  puts "[*] Listening on #{PROXY_HOST}:#{PROXY_PORT}..."

  while true
    # accept conexiunile
    client_socket = proxy.accept
    addr = client_socket.peeraddr
    puts "[*] Accepted connection from #{addr[3]}:#{addr[1]}"

    # thread pentru a gestiona conexiunea
    Thread.new { handle_client(client_socket) }
  end
end

if __FILE__ == $0
  start_proxy
end
```

```
root@debian:/home/diana/Ruby# ruby proxy.rb
[*] Listening on 127.0.0.1:8888...
[*] Accepted connection from 127.0.0.1:36828
Received request from client:
GET / HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Connection: close

Received request:
GET / HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Connection: close

Modifying HTTP request...
Modified HTTP request:
GET / HTTP/1.1
Host: example.com
User-Agent: CustomProxy/5.0
Connection: close

Received response from server:
HTTP/1.1 200 OK
Content-Type: text/plain
Connection: close

Hello from the server!
```